ECG federated learning: three papers, results and next experiments

Nov 4, 2026 · @shivansh mishra

Summary

The first full ECG results support the core claims of Papers 1 and 3, and support Paper 2 only when the server holds a large validation set. All three papers share one federated-learning system and one task: atrial fibrillation (AFib) detection from 12-lead ECG, across 100 label-skewed PTB-XL clients.

Paper	Claim	Status after the first tests
1. Measure after the updates arrive (ECG replication of the TMLR paper “Predict, Measure, or Train?”)	Picking FedAvg or FedYogi each round by validation loss beats either rule used alone	Supported: +0.030 AUPRC-AUC, 4 of 5 seeds; the gain is speed, not the final model
2. Select and train (our method)	Selecting a rule and then training on the server’s validation set beats either step alone	Supported with the full validation set (+0.014, 5 of 5); not supported with 280 records
3. Measurement vs learned controllers	Direct measurement on validation data beats bandit controllers (FedStrategist, SARA) under poisoning, with no learning	Supported vs SARA (5 of 5) and vs the best FedStrategist variant (4 of 5); our method uses more server time

The main gaps are statistical power (5 seeds), a second dataset, SARA’s home setting (attacks that change over time) and the oracle tests. Section 6 lists the experiments that close them; together they need about 2,800 runs, which is why we are asking for a dedicated GPU.

Setup

Every result uses the same data, system and pre-declared protocol, with settings fixed by calibration on separate tuning seeds before any test run.

Data. PTB-XL, split into 100 label-skewed clients (AFib share 0–100%, 5–1,354 records each), 17,441 training records in total. Per seed, the pooled client test splits are halved into V, the server's validation set (2,198 records, about 146 AFib), and T, the reported test set (2,198 records, about 157 AFib). T is never used for a decision. Checks: every label matches PTB-XL, and no patient is in both the train and test split of one client. 3 of 3,854 test patients also appear in another client's training data; they were left in.

System. Two hosts share one plugin package, so a method is written once and runs in both:

- `distributed/`: the deployable system (FastAPI server, Redis, Celery aggregation worker, S3, Docker clients).
- `simple/`: a single-process simulator used for every experiment here.
- Plugins: client selectors (Oort, random); server rules (FedAvg, FedYogi, median, Krum, plus the mechanism controls); controllers (static, select, train, select_train, train_select, random_mix, fedstrategist, sara); client attacks.

Fixed settings. Oort picks 10 of 100 clients per round; 5 local epochs, batch 32, plain SGD clients, a 1-D CNN; FedYogi $\tau = 0.001$, $\beta = 0.9 / 0.99$; one server epoch when training on V.

Metric. AUPRC-AUC, the mean test AUPRC over all rounds. Also reported: late AUPRC-AUC (second half), final AUPRC (last 5 rounds), F2 at threshold 0.3, Brier score and server cost. Comparisons are paired by seed; "4/5" means the method won on 4 of 5 seeds, and p is from a paired t-test.

Protocol. Hypotheses were written before any run (`simple/experiments/ecg_protocol.md`). Tuning used seeds 1–2 for 40 rounds; tests used seeds 42–46 for 60 rounds. SARA was added afterwards as a dated amendment, written before it ran.

Calibration (tuning seeds, test AUPRC-AUC; steps run in order, each fixing its setting for the next):

Step	Settings tried	AUPRC-AUC	Chosen
1. Preprocessing (FedAvg)	none / per-record z-score	0.479 / 0.590	per-record
2. Client learning rate (FedAvg)	0.01 / 0.03 / 0.05 / 0.1	0.396 / 0.540 / 0.590 / 0.628	0.1, the edge of the grid
3. FedYogi step η	0.01 / 0.003 / 0.03	0.627 / 0.541 / 0.598	0.01
4. Server-training step (× client lr)	1.0 / 0.1 / 0.01	0.752 / 0.755 / 0.679	1.0 (within 0.005, the first listed wins)

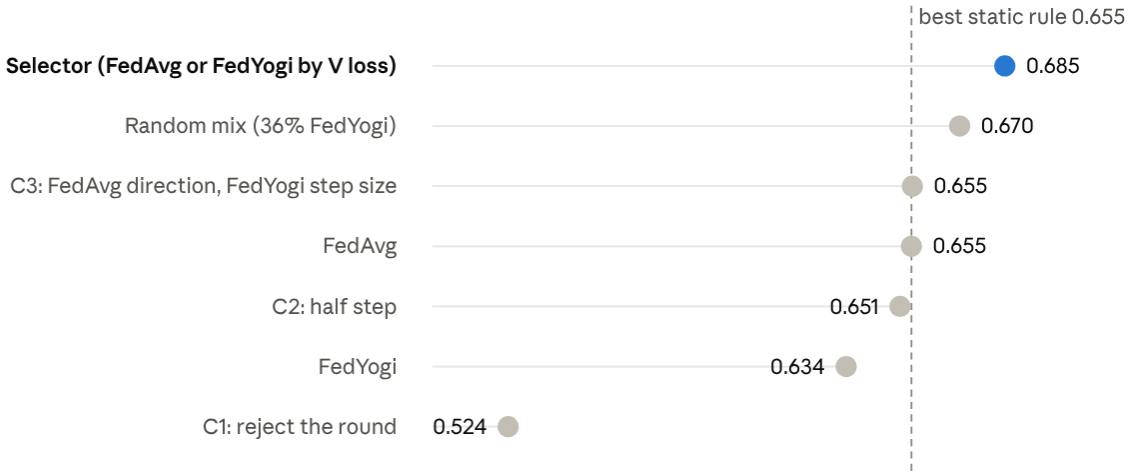
A bug fixed on the way. The TMLR code's counterfactual log recorded all four arms for the selector instead of its two candidates. Our controllers log only the candidates they build, and a regression test guards this.

Paper 1: selection by validation loss

Picking FedAvg or FedYogi each round by validation loss beats both rules used alone, and none of the mechanism controls explains the gain.

The selector beats both static rules; no control matches it

Selector 0.685 vs best static rule (FedAvg) 0.655: +0.030 AUPRC-AUC, mean of 5 seeds



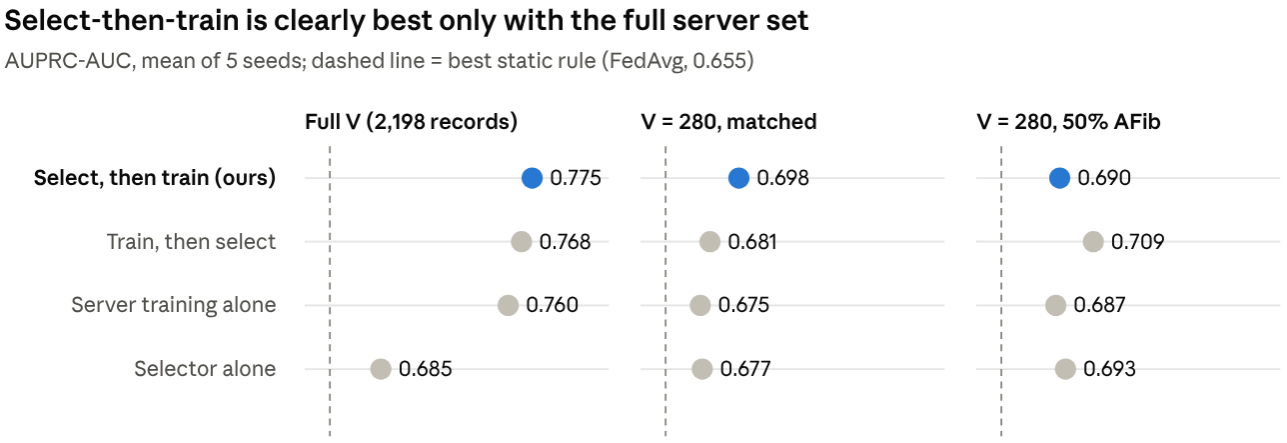
simple/results/ecg/main · mean test AUPRC-AUC over test seeds 42–46, 60 rounds

- **H1, selector vs the per-seed best static rule:** +0.030 ± 0.029 (4/5 seeds, p = 0.04), supported. Over the last 5 rounds the difference is −0.001 (3/5): the selector learns faster but ends in the same place.

- **P1b, mechanism controls:** none reaches 80% of the selector's gain on any seed (gains over the best static rule: reject −0.131, half step −0.004, FedAvg direction at FedYogi's step −0.000). The gain is not just a matter of smaller steps.
- **H1c, selector vs random mixing:** the selector kept FedYogi in 35.7% of rounds; mixing at random at that rate loses by 0.015 ± 0.020 (4/5, p = 0.12). Supported by the pre-declared 4-of-5 rule, but not significant.
- **Cost:** 2 models scored on V per round, 0.32 s of server time.

Paper 2: selection plus server training

Select-then-train is the best method when the server holds the full validation set, but its advantage disappears with 280 records.



simple/results/ecg/{main, v280, v280_afib50} · mean test AUPRC-AUC over test seeds 42–46, 60 rounds

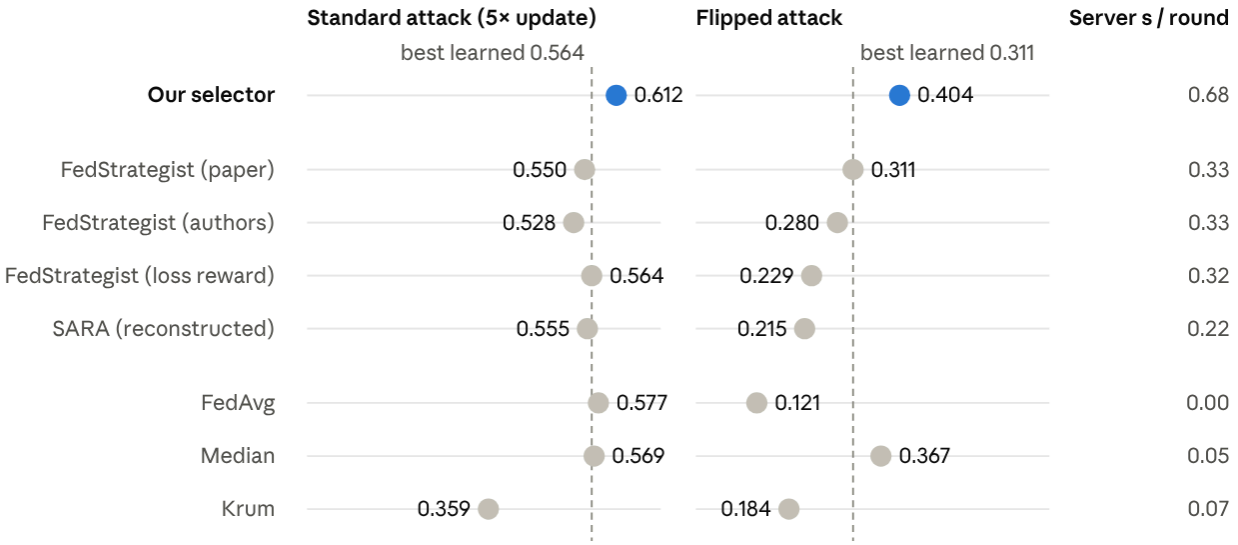
- **H2, select-then-train vs the better of selection alone and training alone:** full V +0.014 ± 0.012 (5/5, p = 0.03), supported; 280 matched +0.011 ± 0.040 (3/5), not supported; 280 at 50% AFib −0.015 ± 0.040 (2/5), not supported.
- **H2r, robustness to a skewed server set:** with 280 records at 50% AFib, select-then-train stays above the best static rule by +0.034 (4/5, p = 0.09), supported.
- **With the full V,** select-then-train is +0.119 above the best static rule (5/5, p < 0.001) and +0.006 above train-then-select (5/5, p = 0.06).
- **Reading:** most of the gain comes from training on V. Selection adds a little on top, and only when V is large.
- **Cost per round with the full V:** select-then-train 1.20 s (2 models scored, 1 training pass); train-then-select 1.62 s (4 scored, 2 passes).

Paper 3: measurement vs learned controllers under poisoning

Under both attacks, our selector (FedAvg, median and Krum scored on V each round) beats every learned controller and every fixed rule, but uses about twice the bandits' server time. Setup: 20 of 100 clients are malicious (fixed per seed), and a selected malicious client uploads 5× its honest update; the flipped attack also reverses its direction. Krum is told how many malicious clients are in the round.

Our selector scores highest under both attacks

AUPRC-AUC, mean of 5 seeds, 20 of 100 clients malicious; dashed line = best learned controller



simple/results/ecg/{poison_standard, poison_flip} · mean test AUPRC-AUC over test seeds 42–46, 60 rounds; server time averaged over both attacks

- **H3, vs the per-seed best FedStrategist variant:** standard +0.038 ± 0.044 (4/5, p = 0.07); flipped +0.056 ± 0.099 (4/5, p = 0.19). Supported by the 4-of-5 rule, not significant.
- **H3s, vs SARA:** standard +0.057 ± 0.039 (5/5, p = 0.015); flipped +0.188 ± 0.115 (5/5, p = 0.011). Supported.
- **Vs the best of all four learned controllers:** +0.028 (4/5, p = 0.15) and +0.056 (4/5, p = 0.19). **Vs the best fixed rule:** +0.030 (4/5, p = 0.06) and +0.037 (4/5, p = 0.13).
- **Why SARA struggles here:** its reward is the model's current quality, which rises for every rule as training goes on, so the bandit can barely tell the rules apart and spreads its picks almost evenly.
- **Cost:** our selector scores 3 models on V per round; the bandits score 1. Paper 3 should argue "no learning and more robust", not "cheaper".

Suggested experiments

Five experiments matter most: more seeds, a second dataset, attacks that change over time, a server-set size sweep and the oracle tests. Each would get its own pre-declared hypothesis in the protocol before it runs.

Priority	Paper	Experiment	Why it matters
High	All	10 seeds instead of 5 for every current test, with Holm correction across hypotheses	Several “supported” results have $p > 0.05$
High	All	A second ECG dataset split by hospital (PhysioNet 2021: CPSC, Chapman-Shaoxing, Georgia), with a larger model	Real differences between sites instead of synthetic label skew; shows the results generalise
High	3	Attacks that switch on and off during training	SARA was designed for this setting; without it the comparison looks unfair to SARA
High	2	Server-set size sweep from 50 to 2,198 records, matched and AFib-skewed	Finds where training on the server set stops beating selection: the paper’s main figure
High	1	Oracle and predictability tests (FedProx client branches)	The TMLR paper’s central claim, still untested on ECG
Medium	3	More attacks (stealth, label flipping, a backdoor, an attacker who knows the selector) and attacker shares of 10–40%	One attack type is not enough for a robustness claim; a backdoor is the expected failure case and should be reported
Medium	3	Trimmed mean, Multi-Krum and FLTrust as rules and baselines	The standard robust baselines reviewers expect
Medium	2	A server set from one hospital, and a separate scoring split for select-then-train	The realistic healthcare case; scoring on the same 280 records used for training may explain the misses at 280
Medium	2	Compare with FLTrust and Zeno; report sensitivity at 90% specificity with the threshold chosen on V	Prior methods that use server data; a metric clinicians read
Medium	1	Rounds needed to reach a target AUPRC, from the existing runs	The selector’s gain is speed, so this is the honest headline

Priority	Paper	Experiment	Why it matters
Medium	1	Selection score: validation loss vs AUPRC vs Brier	An ECG-specific contribution on imbalanced data
Medium	1	Sweep how uneven the clients' data is	Shows where selection helps and where it does not
Low	All	Remove the 3 overlapping test patients; try client lr 0.2 and 0.3	Clean-up; the chosen lr of 0.1 was at the edge of the grid
Low	3	Get the SARA paper (Hu et al., SEAMS 2025) and check our reconstruction	Our SARA is rebuilt from the TMLR paper's description; its "combinations" of rules are missing

System improvements and compute

The planned experiments need about 2,800 runs: about 3 days nonstop on the current laptop GPU, or about 16 days on CPU only. One run is one seed trained for 60 rounds.

Experiment	Runs	Laptop GPU	CPU only
10 seeds for every current test	165	4 h	22 h
Paper 1: oracle tests with FedProx clients	about 120 (assumed 4× cost)	3 h	16 h
Paper 2: server-set size sweep	480	13 h	64 h
Paper 3: more attacks and attacker shares	1,440	38 h	8 days
Second ECG dataset, larger model	about 600 (assumed 5× cost)	16 h	3.3 days
Total	about 2,800	about 3 days	about 16 days

The times scale two measurements from 1 Nov 2026 on the team laptop (RTX 4060 Laptop GPU, 8 GB): about 1.6 minutes per run with two jobs in parallel, and a 5× slowdown on CPU (about 4 s vs 20 s per round). The same seed gave slightly different AUPRC on GPU and CPU, so every run in one paper should come from one machine. The case for a dedicated GPU is in the GPU request slides, also saved in the repo as `docs/gpu_request.pptx`.

System improvements, in order of payoff:

- **A cheaper selector.** Score all candidates on V in one batched GPU pass, or on a stratified subsample of V. Today the selector scores 3 models per round, against 1 for the bandits.
- **Less startup time.** Round 1 takes about 17 s against about 4 s for later rounds; reusing the data and model across seeds would save an estimated 10 h over 2,800 runs.
- **Reproducible runs.** Turn on TensorFlow's deterministic ops, and record the device, git commit and library versions in every results CSV.
- **A systems angle.** Measure the Docker/AWS deployment (round latency, cost, scaling with the number of clients), and run the 49 tests in CI on every push.
- **Done already.** The job runner now prints every round live with overall progress and an ETA.

Caveats and open questions

The results are first-pass evidence: five seeds give wide intervals, and two baselines are reconstructions.

- **Power.** Several "supported" verdicts meet the 4-of-5 rule but have $p > 0.05$ (H1c, H3 under both attacks).
- **Learning rate.** The chosen client lr of 0.1 sits at the edge of the calibration grid; 0.2 and 0.3 were not tried.
- **Patient overlap.** 3 of 3,854 test patients also appear in another client's training data.
- **Missing tests.** The oracle and predictability battery of Paper 1 needs FedProx client branches and has not run.
- **SARA.** The SEAMS paper is paywalled and has no public code. Our version follows the TMLR paper's description: UCB1 over FedAvg, median and Krum, updating only the applied rule, rewarded with accuracy minus loss on V. The paper's "combinations" of rules are not included.
- **FedStrategist.** Ported from our reproduction of the authors' code; we score its reward on V, where the authors used the test set.
- **Krum.** It does badly on these label-skewed clients (0.359 vs 0.577 for FedAvg under the standard attack), because it keeps one client's model.

Open question: should Paper 3 be framed as "no learning and more robust" rather than "cheaper", given that the selector uses about twice the server time of the bandits?

Where things live (repository `SmartMonitoring`):

What	Path
Protocol, calibration and SARA amendment	`simple/experiments/ecg_protocol.md`
Full results report	`simple/results/ecg/REPORT.md` (made by `simple/experiments/analyze_ecg.py`)
Job files and runner	`simple/experiments/jobs_*.txt`, `simple/experiments/run_jobs.py`
Plugins (selectors, rules, controllers, attacks)	`distributed/server/plugins/`
Simulator	`simple/` (`run.py`, `smartmon/`)
GPU request slides	`docs/gpu_request.pptx`